Documentación Técnica — Proyecto Fase 3: Migración a NoSQL (MongoDB)

Universidad de San Carlos de Guatemala (USAC)

Facultad de Ingeniería — Escuela de Ciencias y Sistemas

Curso: Sistemas de Bases de Datos 2 (Segundo Semestre 2026)

Grupo : 14

Grupo14	Marcelo Andres Juarez Alfaro	202010367
	Carlos David de León Barrios	202112109
	Angel Rafael Barrios González	202300733

1. Introducción y Herramientas Utilizadas

Esta fase del proyecto documenta la migración de la información histórica de los Juegos Olímpicos desde un modelo relacional estricto (SQL Server, desarrollado en las Fases 1 y 2) hacia un modelo orientado a documentos en **MongoDB**. El objetivo principal es estructurar la base de datos para optimizar los tiempos de respuesta en consultas de lectura intensiva, cumpliendo con los requerimientos de búsqueda por atleta y por país.

Stack Tecnológico:

- **Origen de Datos:** Microsoft SQL Server (Instancia limpia `OlimpiadasDB` consolidada en las fases anteriores).
- **Motor NoSQL Destino:** MongoDB Community Server 9.0.4 y MongoDB Shell (`mongosh` 2.13.0).
- **Gestión y Transformación (ETL):** Python 3.14.0, utilizando las librerías `pandas` (para manipulación de DataFrames), `SQLAlchemy` y `pyodbc` (extracción desde SQL Server), y `pymongo` (conexión, transformación BSON e inserción en MongoDB).

2. Decisiones de Diseño y Justificación de la Migración

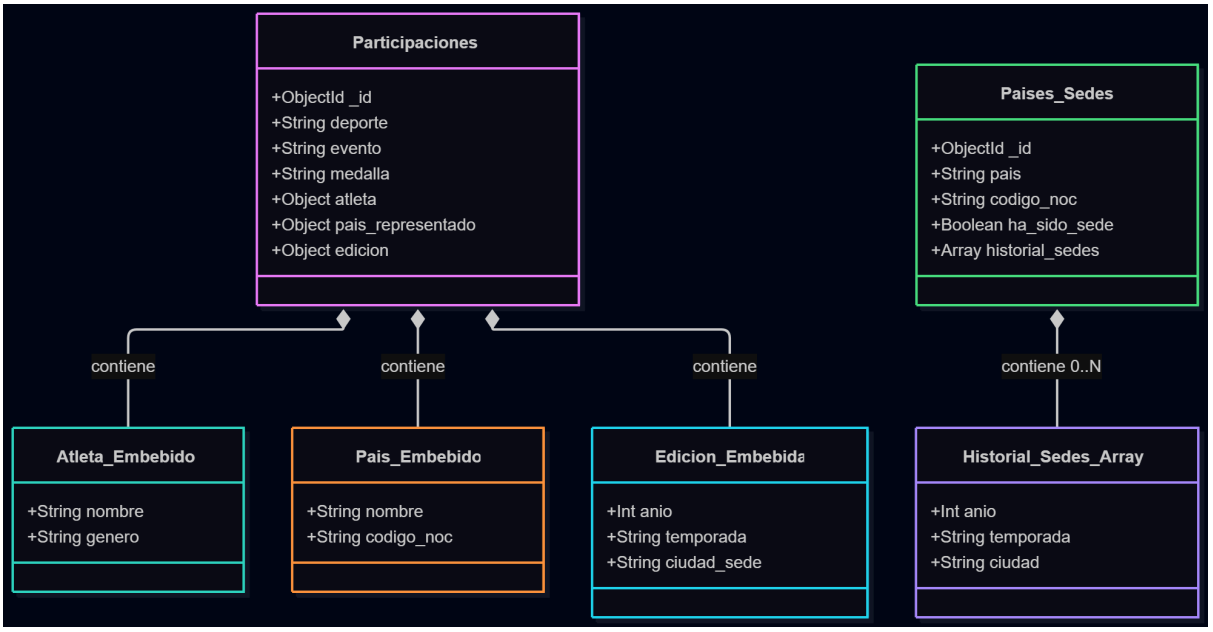
El cambio de un paradigma SQL a NoSQL requiere abandonar la normalización en favor de la **desnormalización** para favorecer el rendimiento de lectura. En la base de datos relacional, la información estaba fragmentada en 9 tablas (`ATLETA`, `PAIS`, `PARTICIPACION`, `EVENTO`, `EDICION_JUEGOS`, etc.), lo que exigía múltiples operaciones `INNER JOIN` costosas a nivel computacional.

Decisiones Arquitectónicas Clave:

1. **Documentos Embebidos (Colección `Participaciones`):** Se decidió agrupar toda la jerarquía de una participación olímpica en un solo documento JSON. Cada documento contiene ahora sub-documentos para `atleta`, `edicion` y `pais_representado`. Esto permite que el script de búsqueda por atleta recupere toda la información de un solo viaje a la base de datos, sin cruzar colecciones.
2. **Uso de Arreglos para Relaciones 1:N (Colección `Paises_Sedes`):** Para resolver la interrogante de si un país ha sido sede y en qué años, se creó una colección secundaria. En lugar de tener una tabla relacional de sedes, se utilizó un arreglo (`historial_sedes`) dentro del documento de cada país, almacenando ahí los sub-documentos con el año y la ciudad.
3. **Limpieza Nativa en NoSQL:** Aprovechando el motor de expresiones regulares de MongoDB (`$regex`), se ejecutó una limpieza post-migración para eliminar 85,264 participaciones duplicadas heredadas del dataset original (`results.csv`) que contenían el sufijo "(Olympic)", garantizando la pureza de los datos exigida en la Fase 3.
4. **Indexación Estratégica:** Se crearon índices de texto plano en los campos `atleta.nombre` y `pais_representado.nombre` para asegurar tiempos de consulta en el orden de los milisegundos.

3. Modelo de Datos Documental (Esquema BSON)

A diferencia de un Diagrama Entidad-Relación (ER) tradicional, en MongoDB modelamos la estructura jerárquica de los documentos.

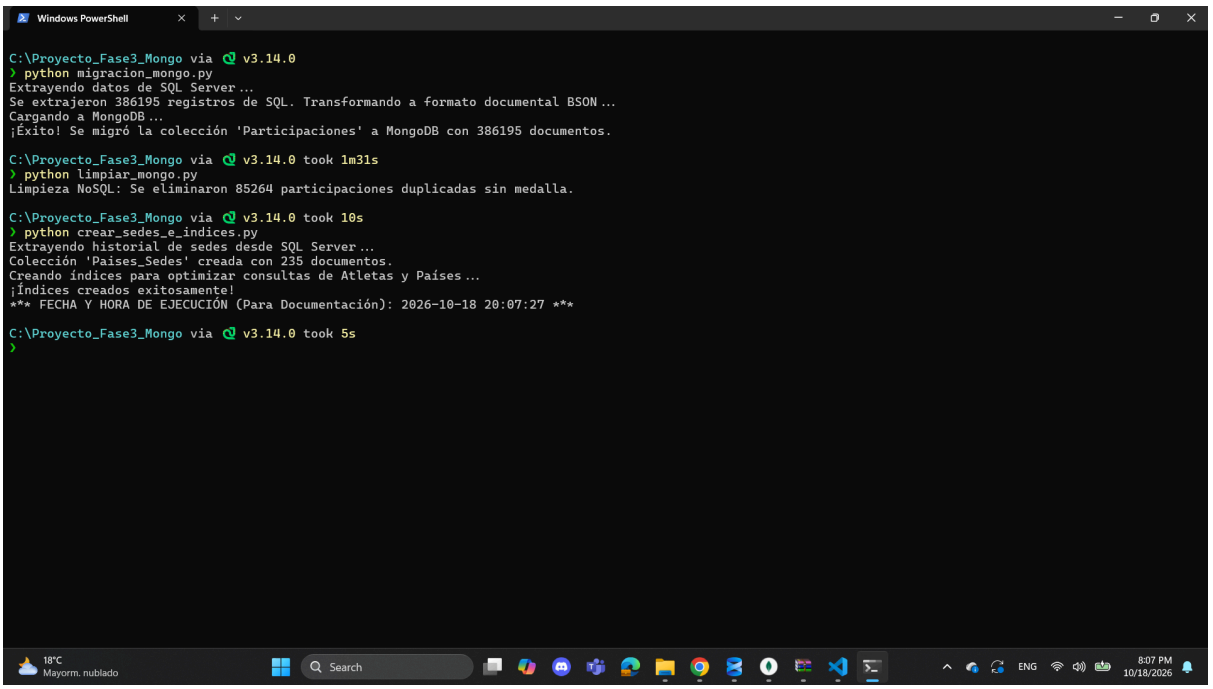


4. Estructura de Scripts y Ejecución de la Fase 3

El proceso de migración y consulta se modularizó en 5 scripts de Python, ejecutados secuencialmente desde la interfaz de línea de comandos (CLI).

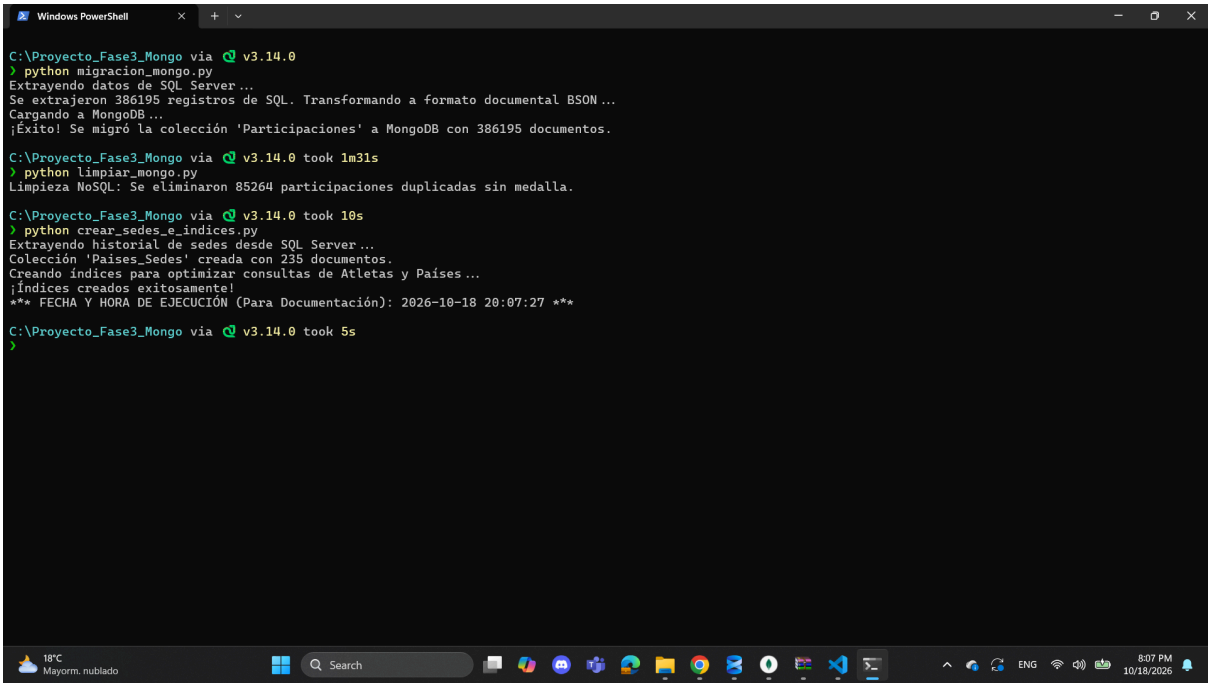
4.1 Extracción y Carga Masiva (`migracion_mongo.py`)

Este script se conecta a la instancia de SQL Server, extrae la data relacional mediante un `JOIN` global y estructura los 386,195 registros en formato BSON para insertarlos mediante `insert_many()` en la colección `Participaciones` de MongoDB.



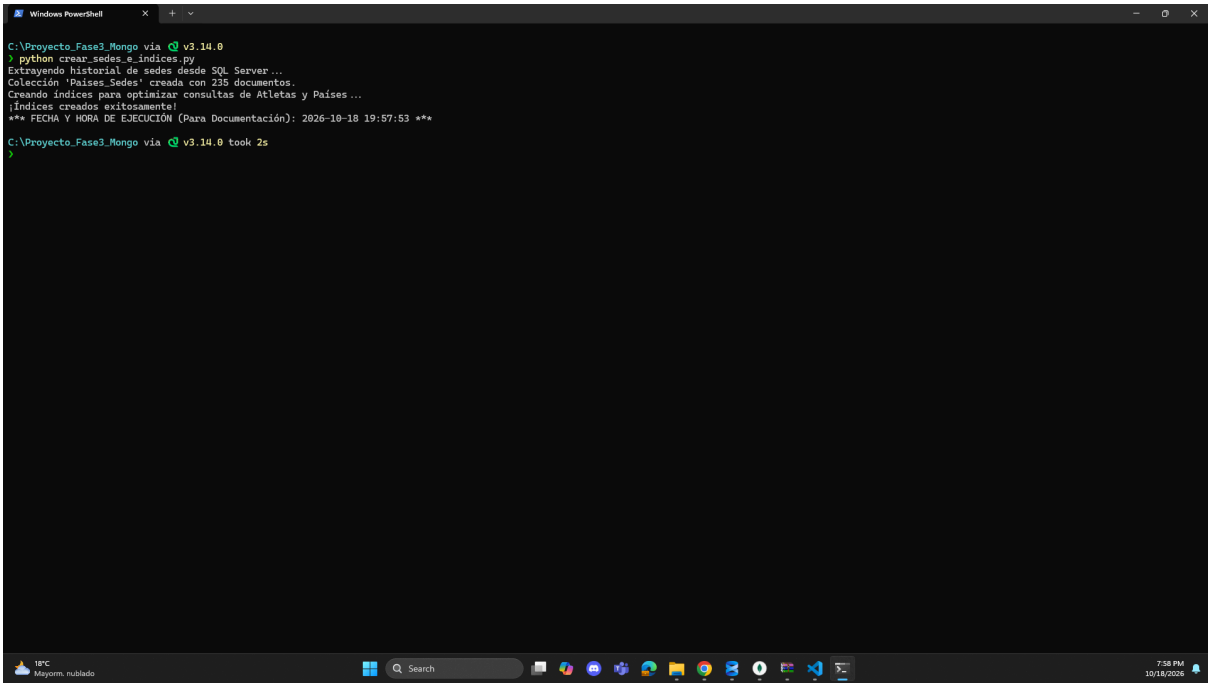
4.2 Limpieza de Datos Post-Migración (`limpiar_mongo.py`)

Encargado de subsanar inconsistencias heredadas de las fuentes originales (como el archivo `results.csv`). Ejecuta una operación `delete_many()` buscando eventos con la etiqueta `(Olympic)` que no hayan resultado en medalla, eliminando 85,264 registros basura para dejar el historial perfecto.



4.3 Creación de Colección de Sedes e Índices (`crear_sedes_e_indices.py`)

Genera la colección `Paises_Sedes` procesando 235 documentos e incrustando el historial de sedes. Además, ejecuta `create_index()` sobre los nombres de los atletas y países, registrando la fecha y hora exacta de ejecución para auditoría del proyecto.

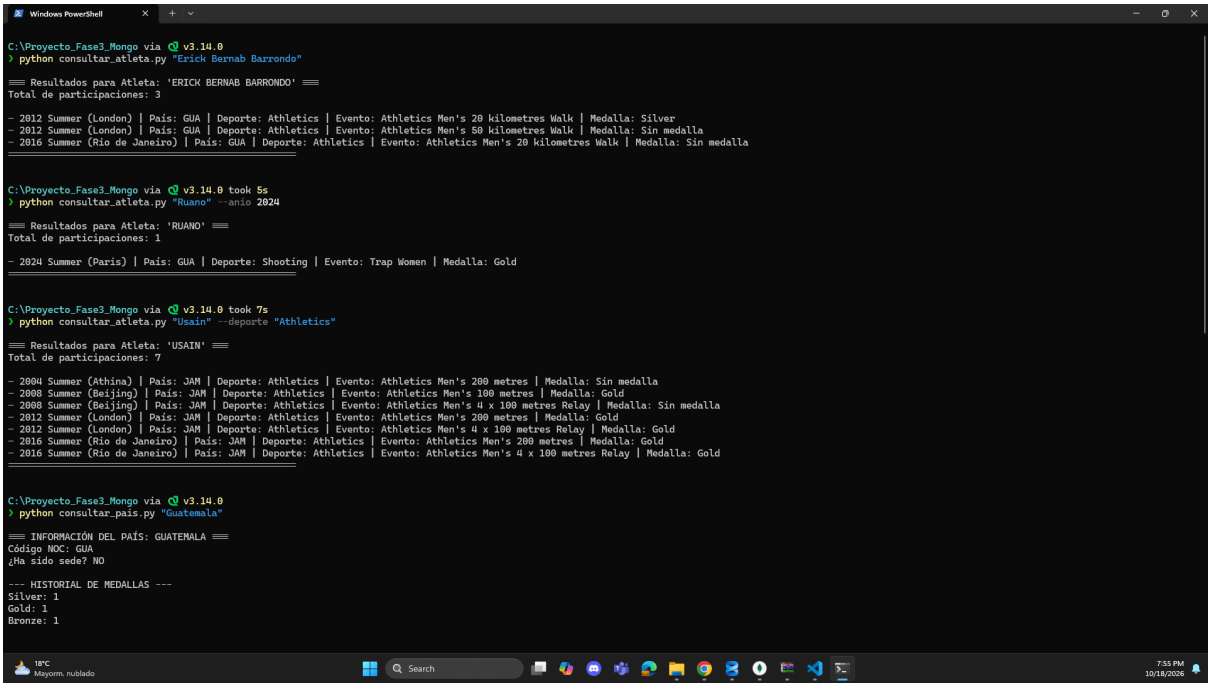


4.4 Consultas Dinámicas desde CLI

Se construyeron dos scripts que utilizan `argparse` para recibir parámetros por consola, armar los filtros dinámicos con los operadores de MongoDB (`$regex`, `$ne`, etc.) y utilizar *Aggregation Pipelines* para totalizar medallas.

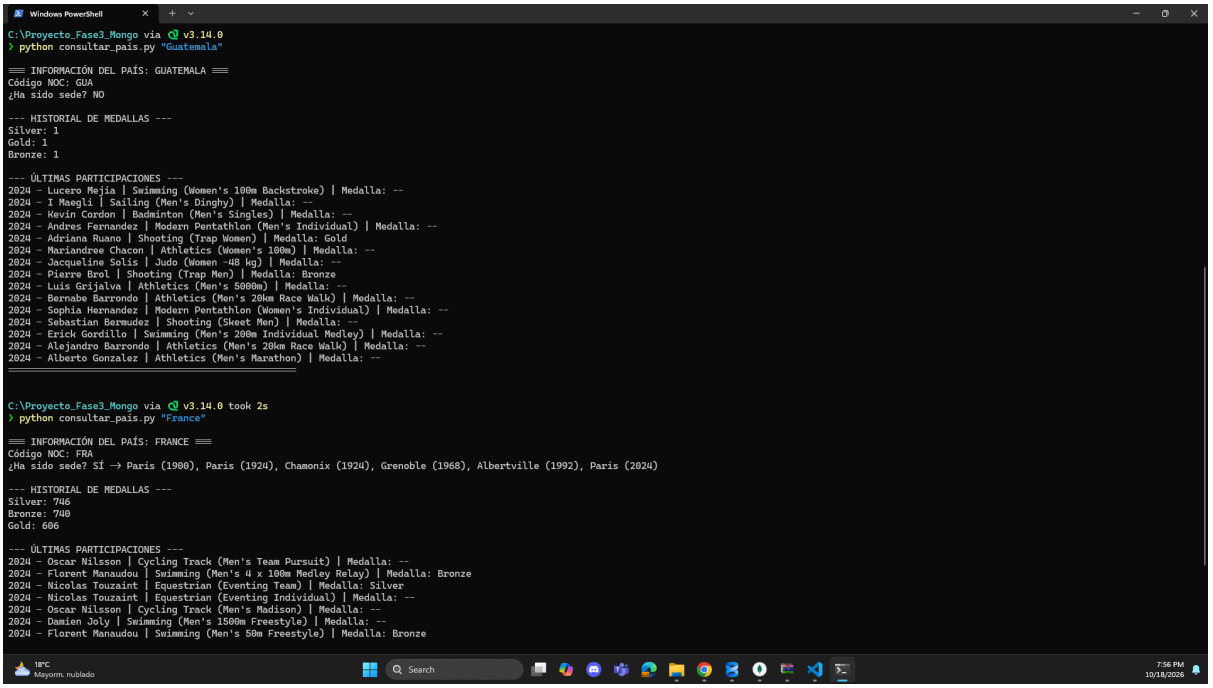
A) Script `consultar_atleta.py`

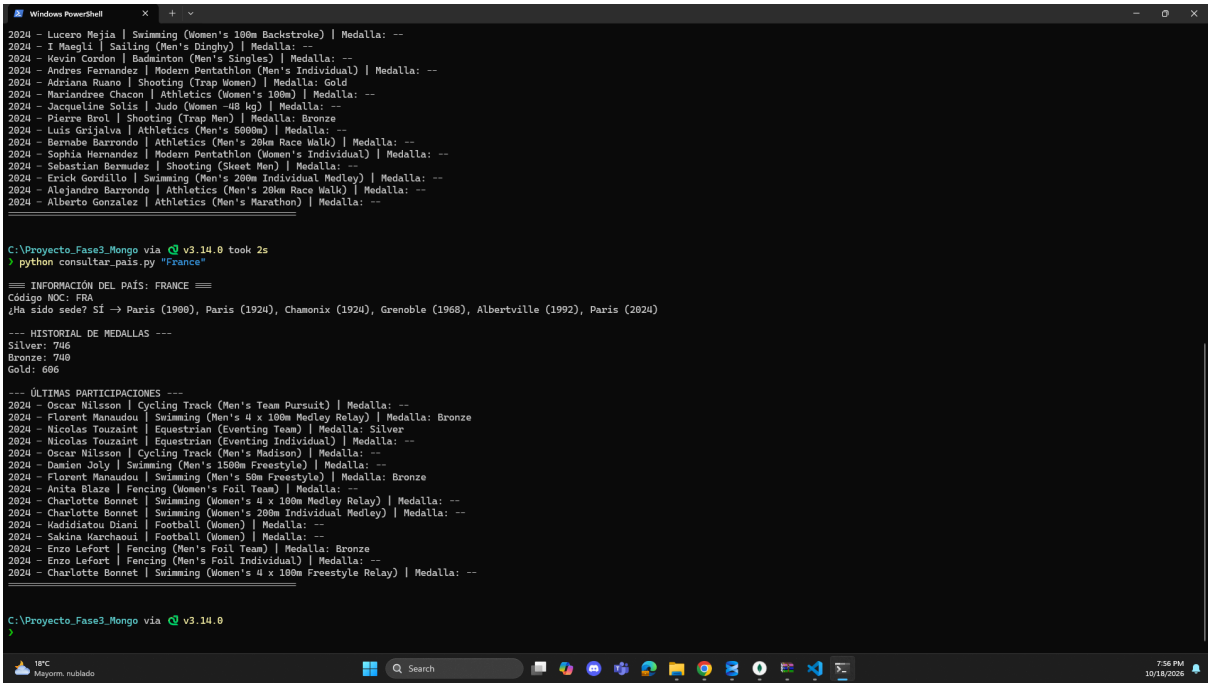
Filtra la colección `Participaciones` por nombre (ignorando mayúsculas/minúsculas) y acepta parámetros adicionales (`--deporte`, `--pais`, `--anio`).



B) Script `consultar_pais.py`

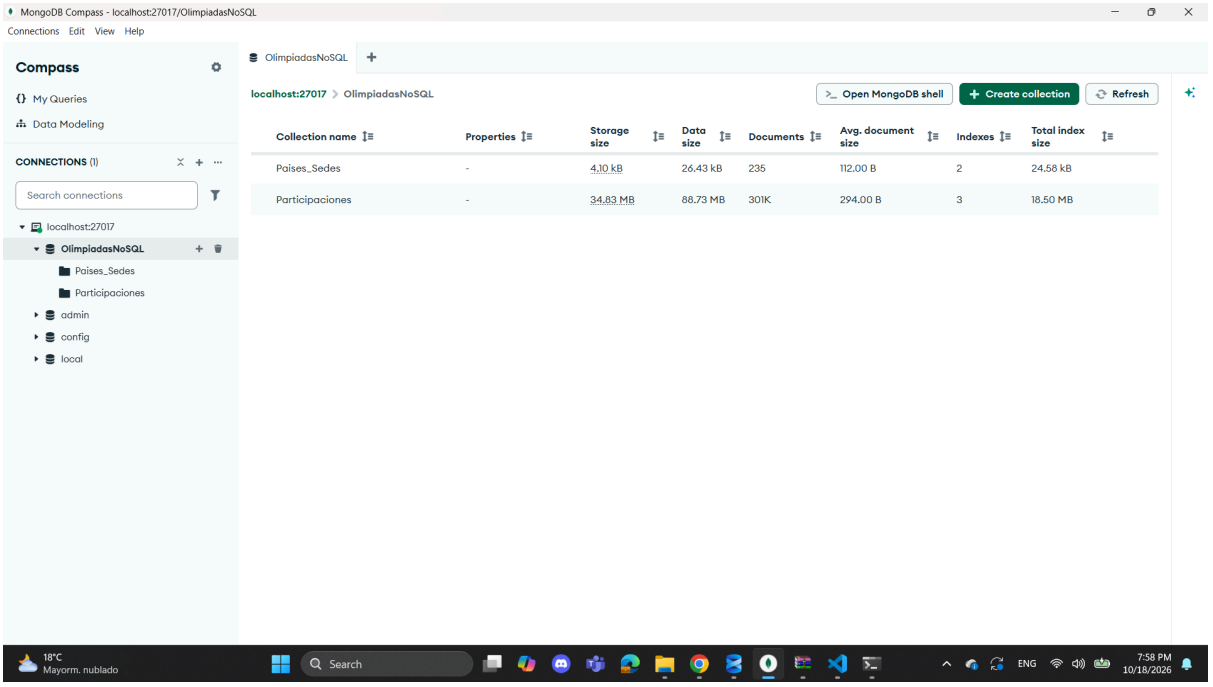
Consulta la colección `Paises_Sedes` para desplegar el historial logístico, y luego utiliza un pipeline de agregación (`$match` y `$group`) sobre la colección `Participaciones` para calcular el conteo histórico de medallas de oro, plata y bronce.

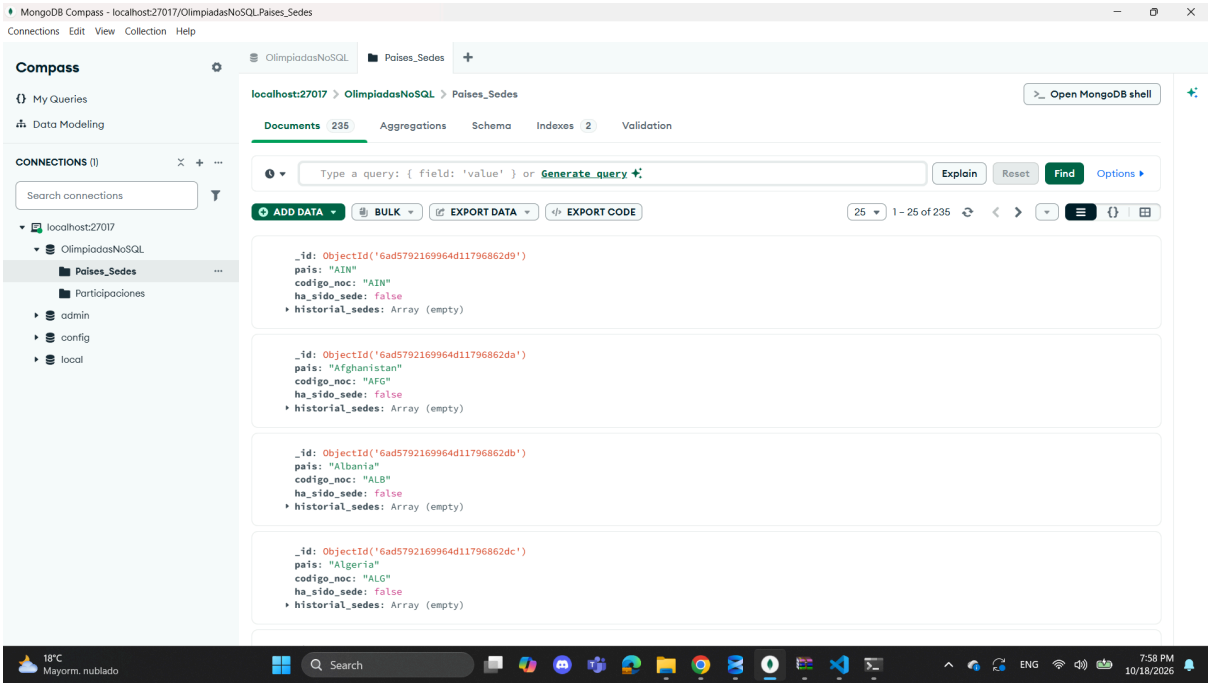


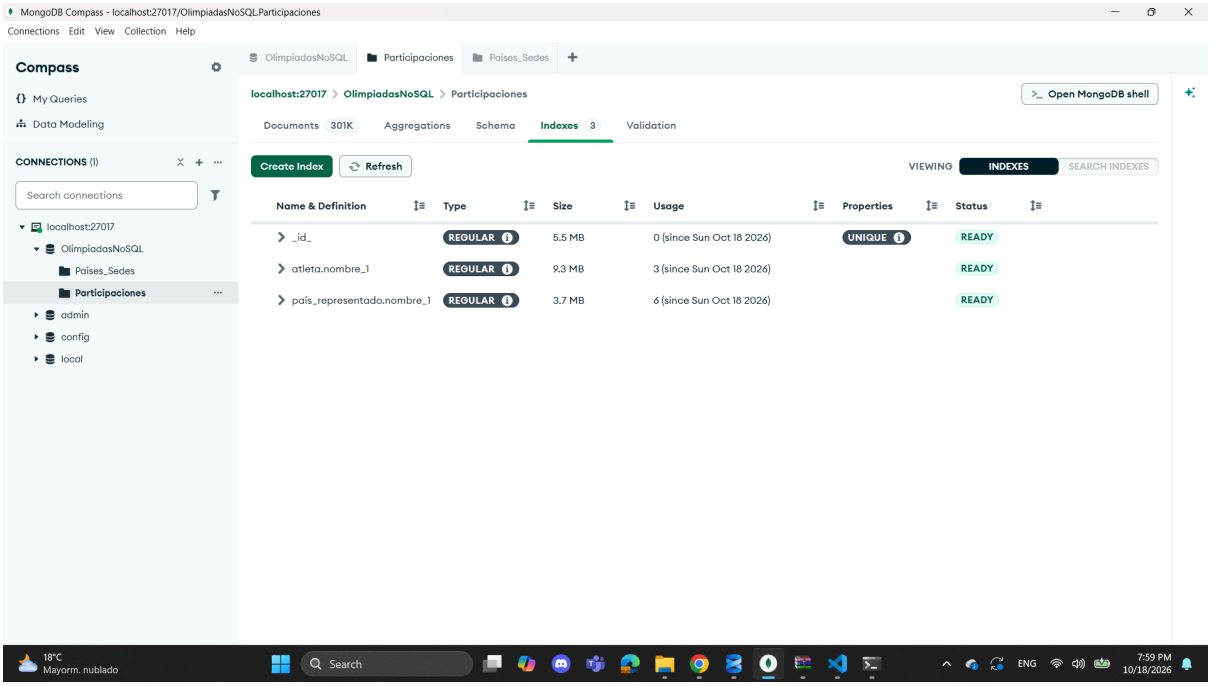


5. Verificación Estructural en MongoDB Compass

Como evidencia final del almacenamiento documental, se inspecciona la base de datos `OlimpiadasNoSQL` a través de la interfaz gráfica de MongoDB Compass, validando la creación de las dos colecciones y el registro físico de los índices creados por el motor.







La flexibilidad de MongoDB ha permitido condensar de forma natural atributos que en SQL requerían uniones pesadas, resultando en un sistema de consultas directo y altamente escalable para el volumen de datos de los Juegos Olímpicos.